



Beata Zalewa

Dos and Don'ts for Azure Serverless
with Azure Functions and Logic Apps



Expert Summit 2020

All about Microsoft® technologies

ONLINE

Sala Technologii

Godz. 13:00

lingaro

GFT

ZALNET



DC DEVELOPER COUNCIL

accenture

kursySQL.pl

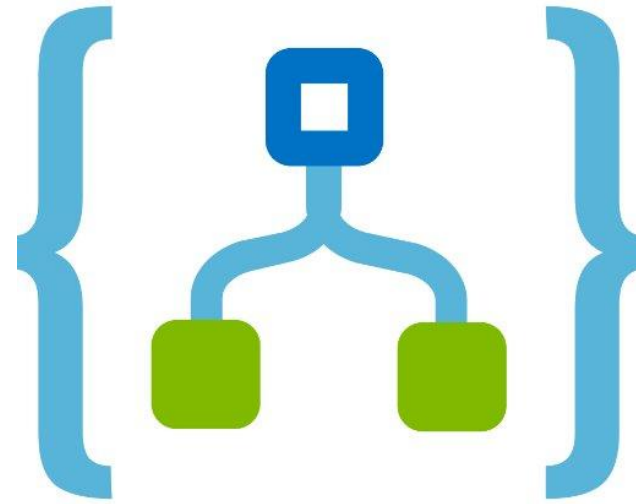
EUVIC:
THE GOOD People

Luxoft
A DXC Technology Company

spark-IT

DIGITAL
DEXTERITY

IT-DEV



Dos and Don'ts for Azure Serverless with Azure Functions and Logic Apps

Beata Zalewa, Expert Summit 2020, 05.11.2020

ABOUT



In professional life consultant, Microsoft Certified Trainer, administrator, developer, freelancer, sometimes a miracle-worker and magician.



From the beginning of career associated with Microsoft technologies.



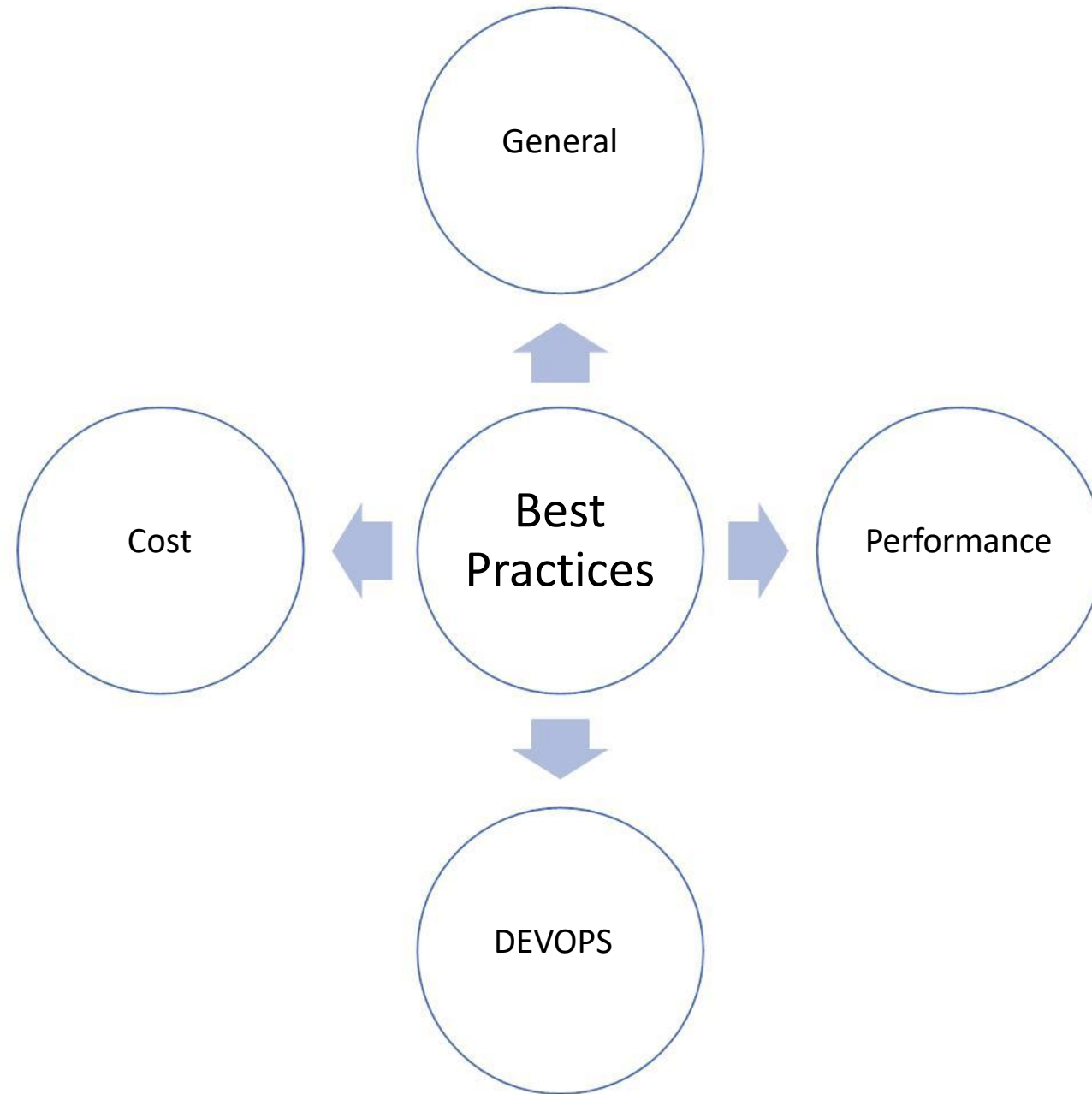
Private life in numbers: 1 husband, 1 daughter, 1 cat and 2 dogs. My hobbies are detective stories and photography.



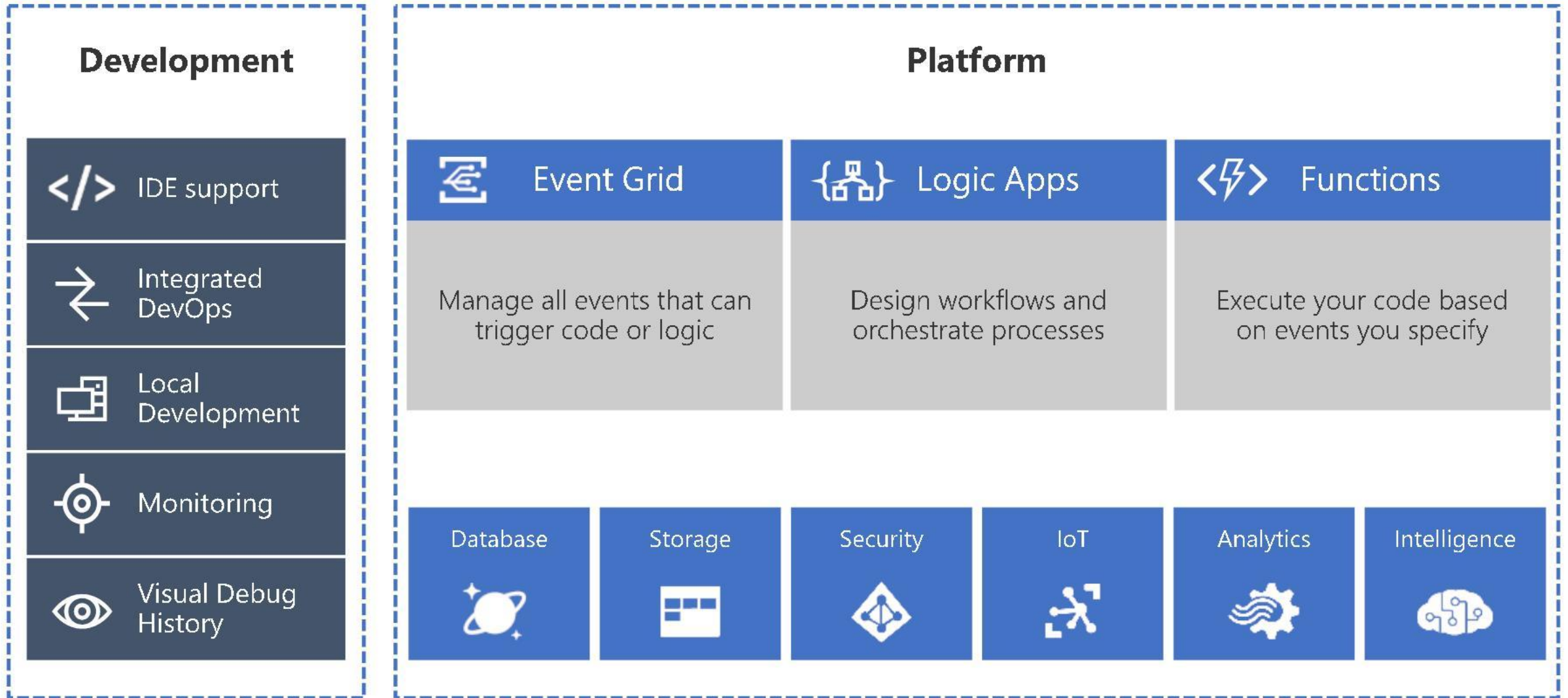
Email: info@zalnet.pl

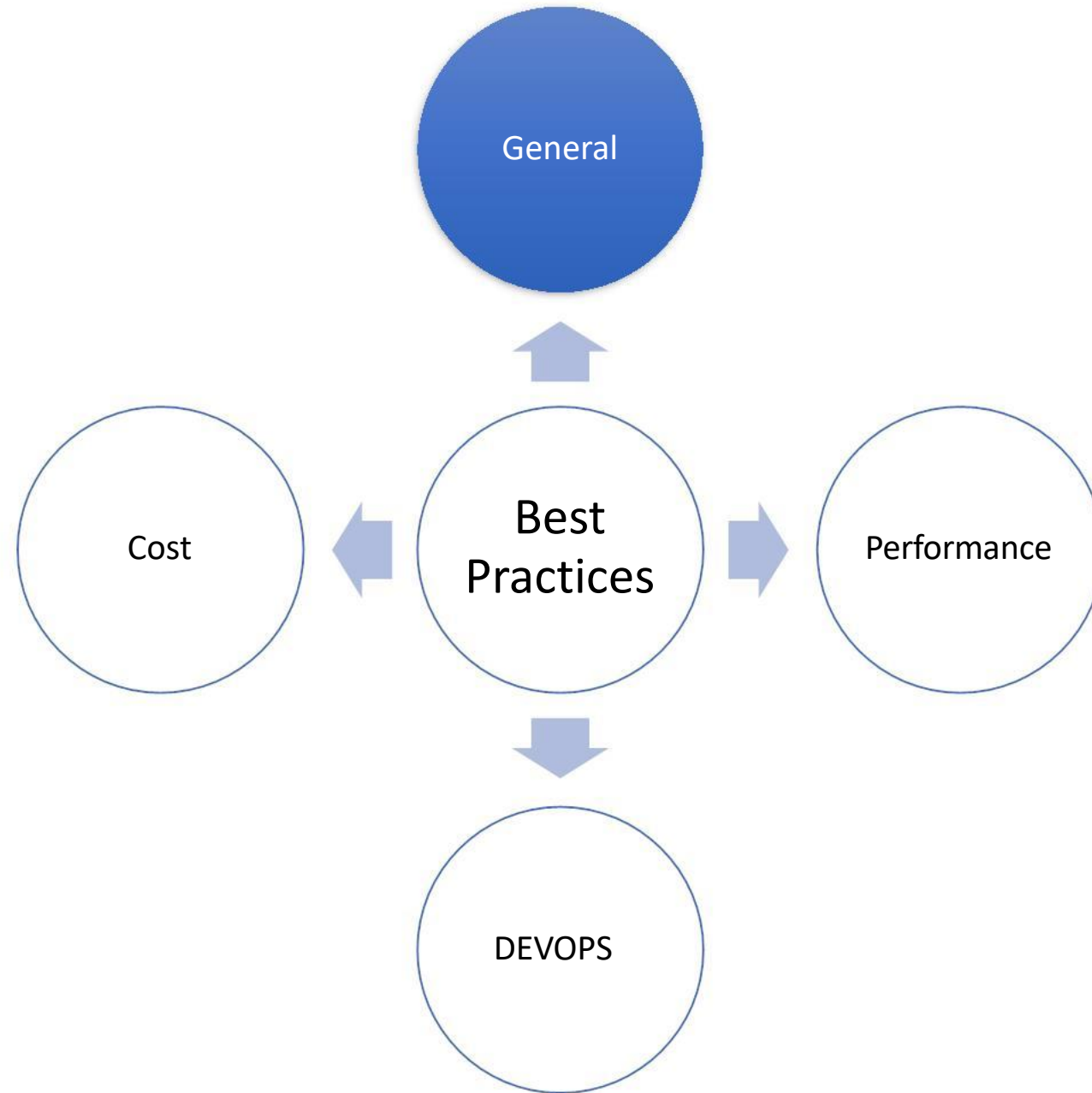


WWW: <https://zalnet.pl> <https://zalnet.pl/blog/>



Serverless platform components





Azure Functions



Solution for running small pieces of code, or "functions," in the cloud:

- Write only code that is relevant to business logic

- Removes the necessity to write “plumbing” code to connect or host application components

Build on open-source WebJobs code

Supports a wide variety of programming languages, for instance, even supports scripting languages, such as:

Subscription * ⓘ

Resource Group * ⓘ

Instance Details

Function App name *

Publish *

Runtime stack *

.NET Core

Node.js

Python

Java

PowerShell Core

Custom

Select a runtime stack



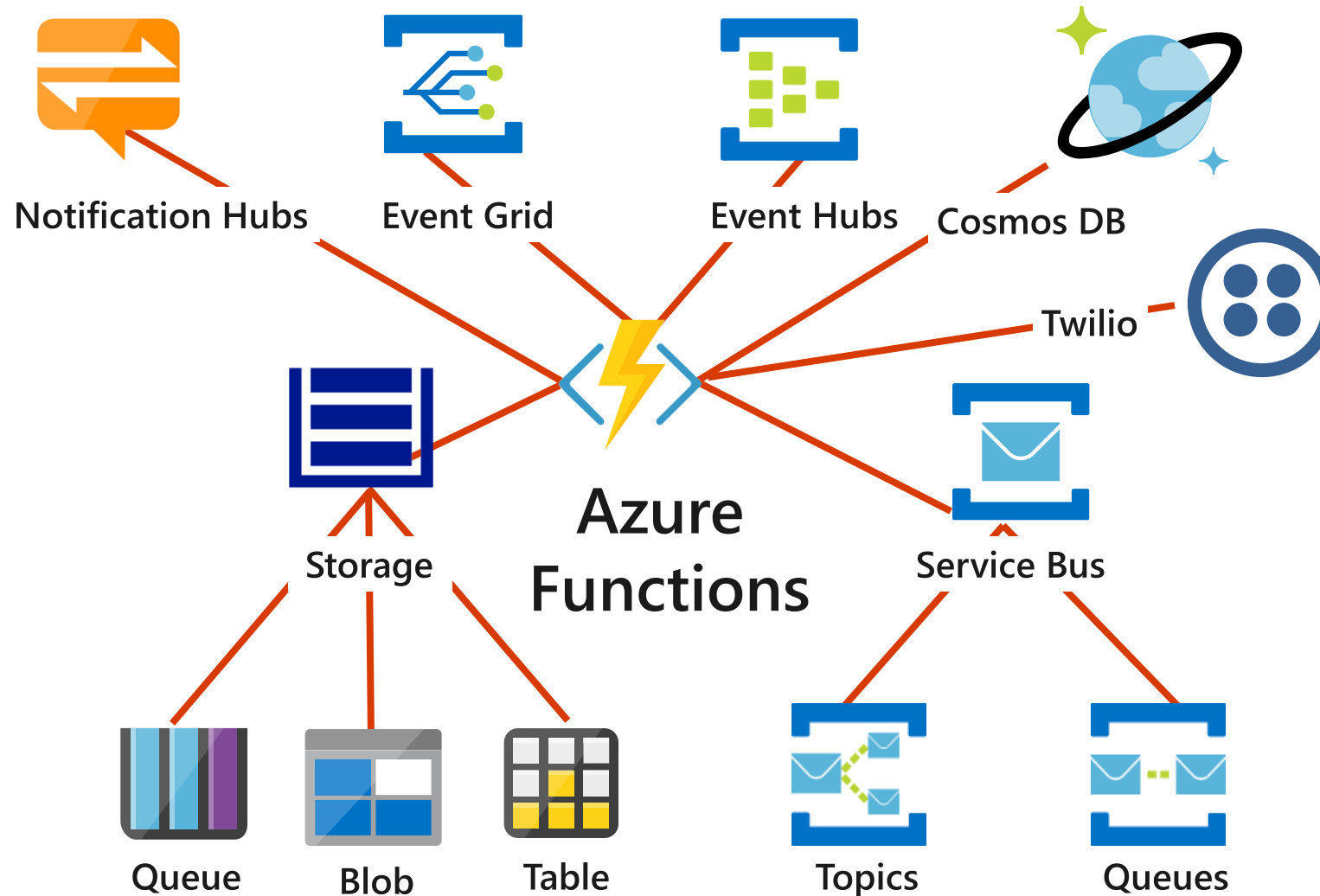
What can Azure Functions do?

- Run code based on HTTP requests
- Schedule code to run at predefined times
- Process new and modified:
 - Azure Cosmos DB documents
 - Azure Storage blobs
 - Azure Queue storage messages
- Respond to Azure Event Grid events by using subscriptions and filters
- Respond to high volumes of Azure Event Hubs events
- Respond to Azure Service Bus queue and topic messages

Trigger types

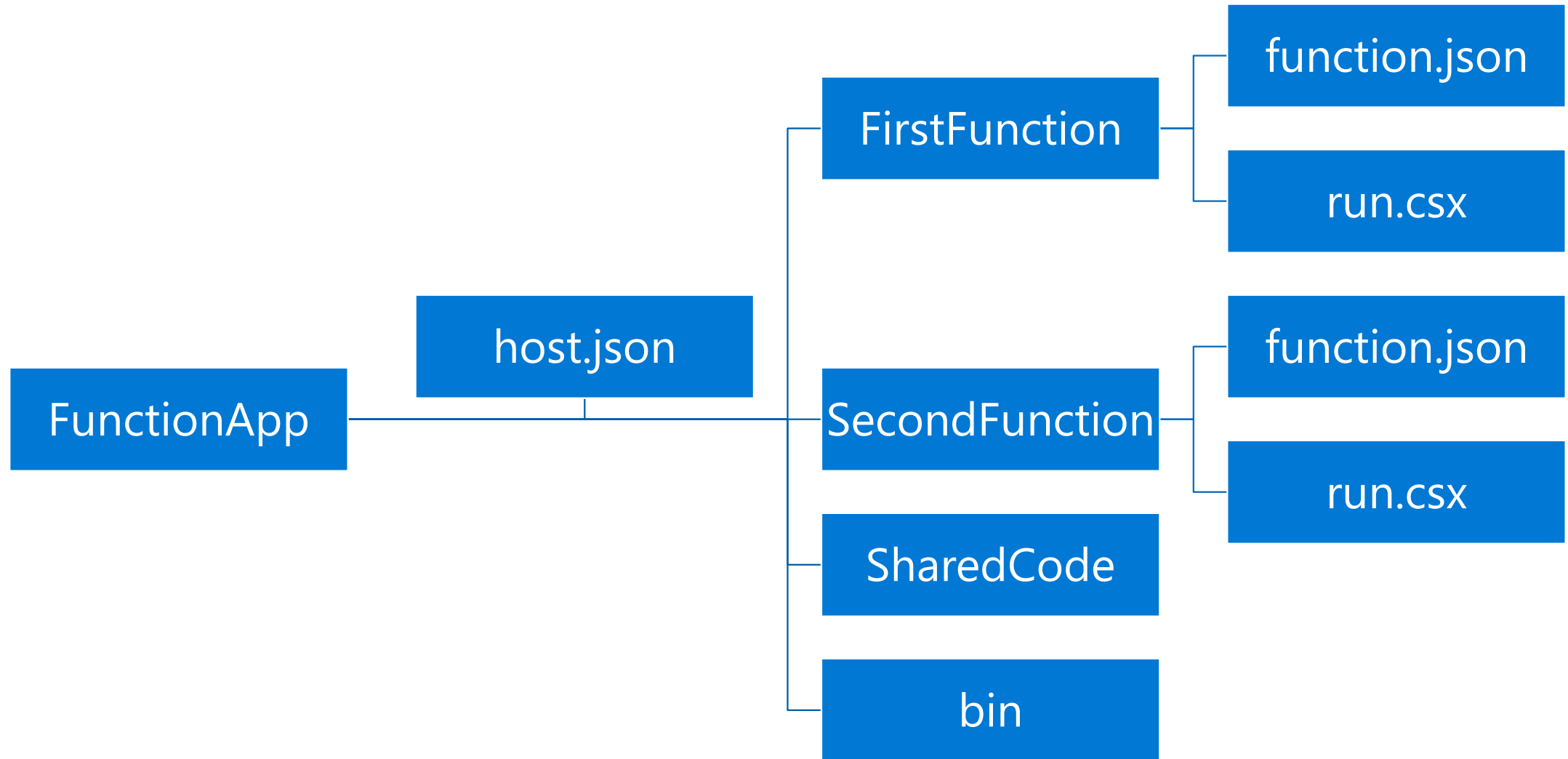
- Triggers based on Azure services:
 - Cosmos DB
 - Blob and queues
 - Service Bus
 - Event Hub
- Triggers based on common scenarios:
 - HTTP request
 - Scheduled timer
- Triggers based on third-party services:
 - GitHub
- And more...

Function integrations



DEMO 1

Function folder structure

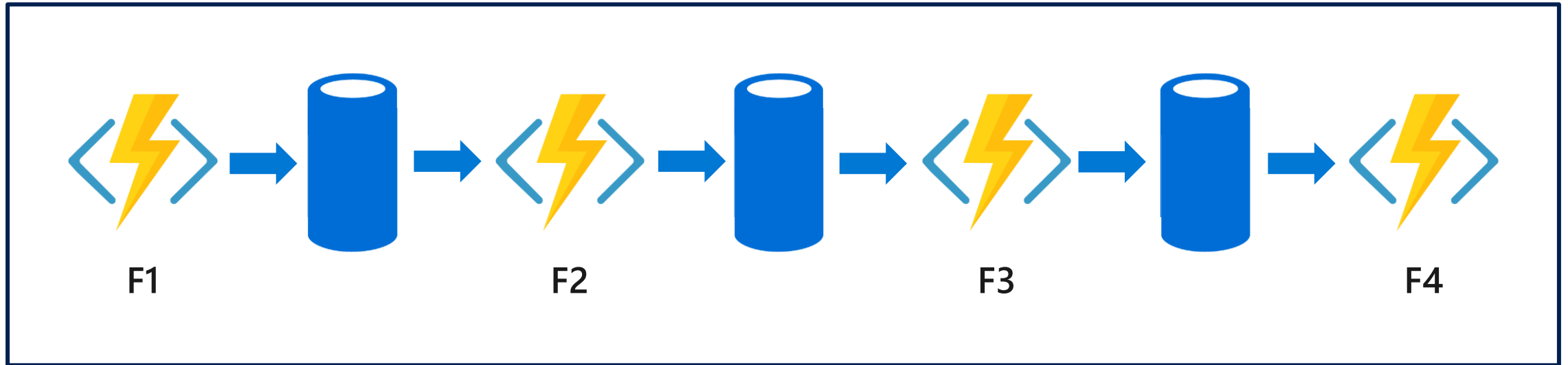


Durable Functions

- Write stateful functions in a stateless environment
- Manages state, checkpoints, and restarts
- Defines an Orchestrator function
 - Workflows are defined in code
 - Calls other functions synchronously or asynchronously
 - Checkpoint progress whenever function awaits

Durable Function scenario - Chaining

Function chaining refers executing a sequence of functions in a particular order. Often, the output of one function needs to be applied to the input of another function.



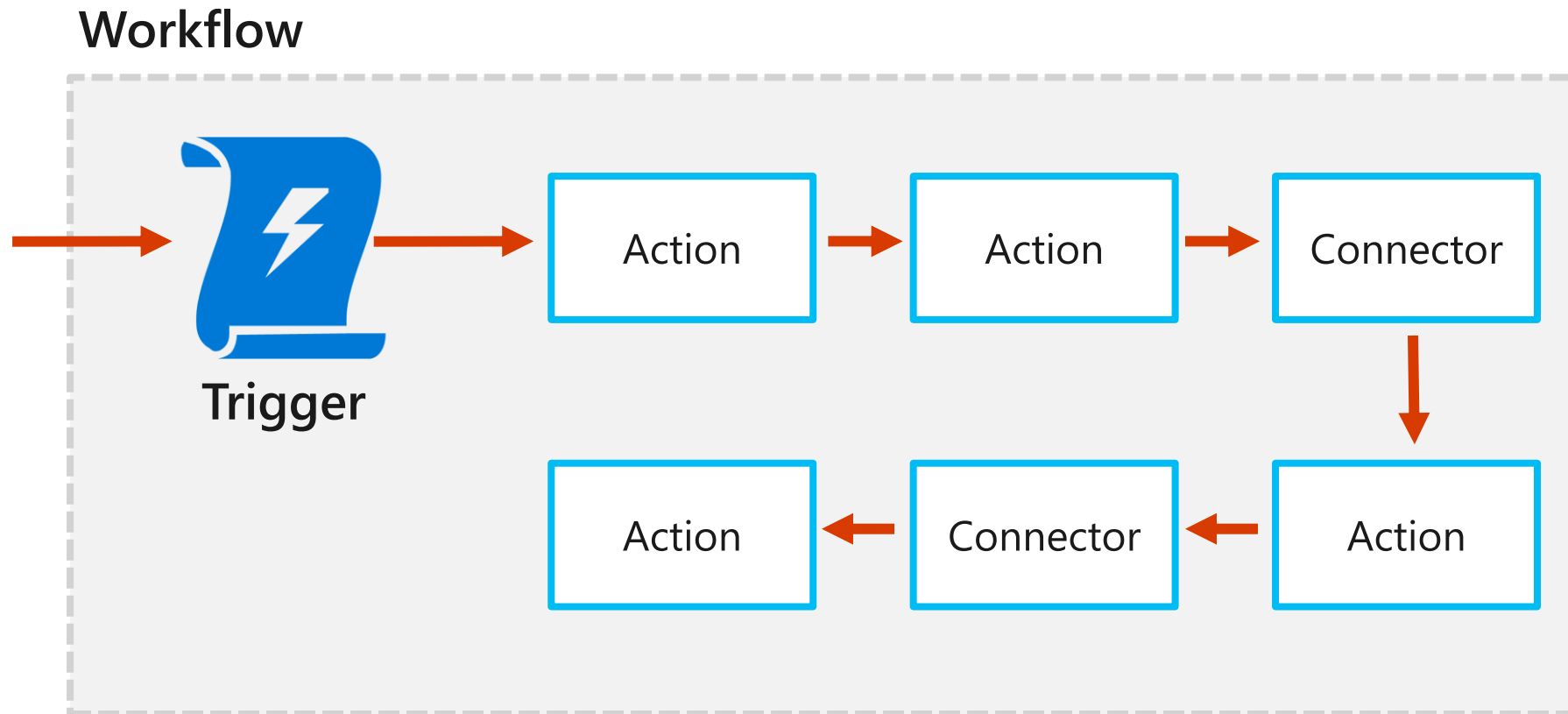
Azure Logic Apps

- Automation workflow solution:
 - No-code designer for rapid creation of integration solutions
 - Prebuilt templates to simplify getting started
 - Out-of-box support for popular software as a service (SaaS) and on-premises integrations
 - BizTalk APIs available to advanced integration solutions
- JSON-based workflow definition:
 - Can be deployed by using Azure Resource Manager templates

Components

- **Workflow**
 - The business process described as a series of steps
- **Triggers**
 - The step that invokes a new workflow instance
- **Actions**
 - An individual step in a workflow, typically a Connector or custom API app
- **Connectors**
 - A special case of an API app that is prebuilt and ready to integrate with a specific service or data source. For example:
 - Twitter and SQL Server Connectors

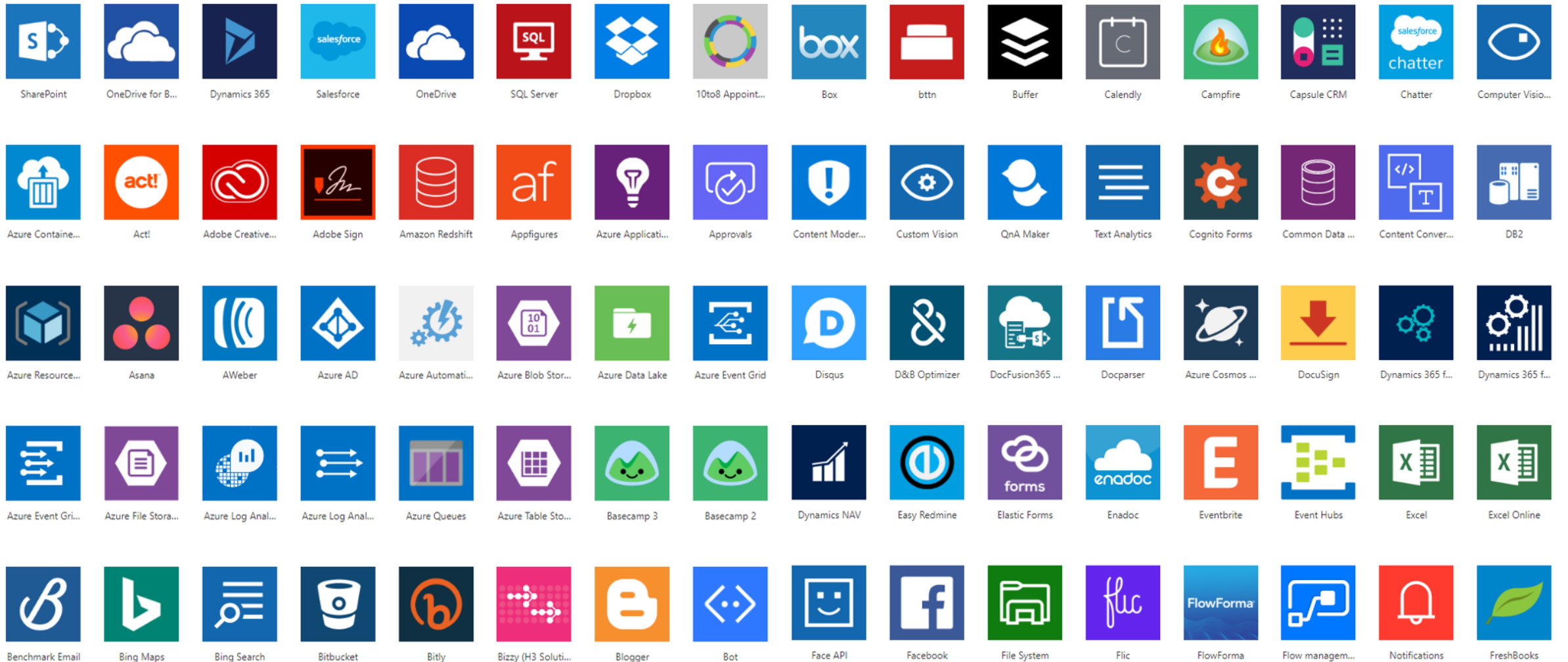
Workflow components



Connectors



Connector ecosystem



Logic Apps Designer – action configuration

The screenshot displays the Logic Apps Designer interface. At the top, a trigger action 'When a feed item is published' is shown in an orange box. A downward arrow points to the 'Send an email' action, which is highlighted in a blue box. The 'Send an email' action configuration is as follows:

- To:** <your-email-address@domain>
- Subject:** New RSS item: Feed title x
- Body:**
 - Title: Feed title x
 - Date published: Feed publishe... x
 - Link: Primary feed l... x

A red rectangular box highlights the 'Body' section, specifically the three dynamic content fields: 'Title', 'Date published', and 'Link'.

Logic Apps Designer – dynamic content

The screenshot displays the Logic Apps Designer interface. At the top, a trigger action 'When a feed item is published' is shown. Below it, an action 'Send an email' is configured. The 'Subject' field contains the text 'New RSS item:'. A red arrow points from the 'Add dynamic content' button in the 'Subject' field to the 'Dynamic content' pane on the right. This pane lists various dynamic content options available for the flow, including 'categories - Item', 'Feed copyright information', 'Feed ID', 'Feed published on', 'Feed summary', and 'Feed title'. The 'Feed title' option is highlighted with a red box.

When a feed item is published

Send an email

* To: <your-email-address@domain>

* Subject: New RSS item: Add dynamic content

* Body: Specify the body of the mail

Show advanced options

Connected to <your-email-address@domain> Change connection.

Add dynamic content from the apps and connectors used in this flow. Hide

Dynamic content Expression

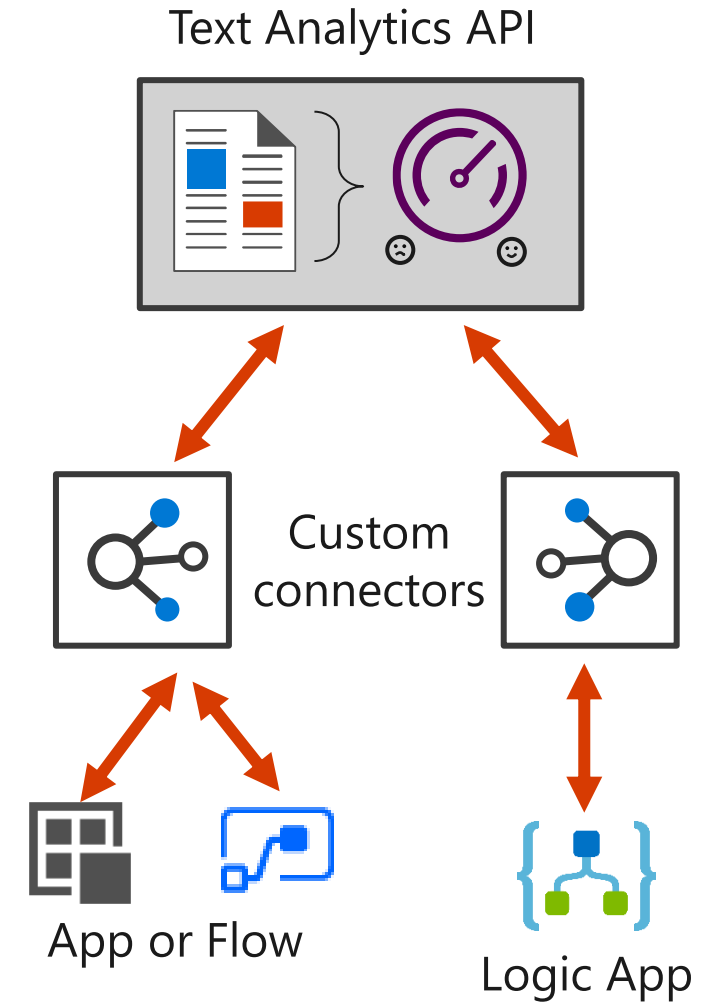
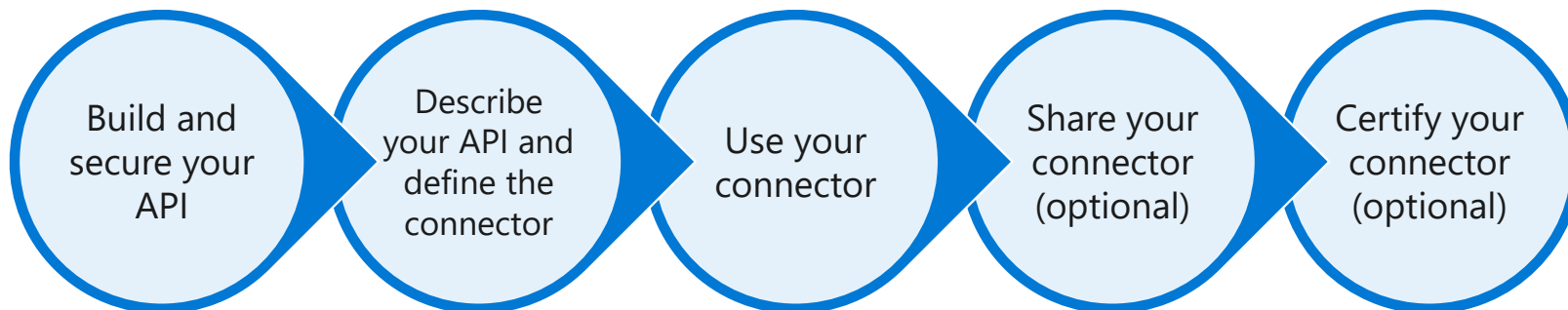
Search dynamic content

When a feed item is published

- categories - Item
- Feed copyright information
Copyright information
- Feed ID
Feed ID
- Feed published on
Feed published date
- Feed summary
Feed item summary
- Feed title**
Feed title

Custom connectors

- Some APIs, services, and systems are available by using prebuilt connectors
- Build custom connectors:
 - Function-based
 - Custom defined triggers and actions



DEMO 2

General Best Practices

- Use descriptive names
- Improves readability
- Transfer knowledge
- You can't rename Logic Apps ... directly
- You can't rename actions that have dependencies ... directly (you can rename it in code)
- If needed, add comments
- Learn from failures

General Best Practices

Home > AzureLogiAppWithFunctionRG >

 **logic-app-h2hpupxtsfidy** 

Logic app

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Development Tools

Logic app designer

 Run Trigger  Refresh  Edit  Delete  Disable  Update Schema  Clone  Export

 To improve traffic flow, we're adding new outbound IP addresses for Logic Apps. Review action needed if you're filtering IP addresses with firewall se

^ Essentials

Resource group [\(change\)](#) : AzureLogiAppWithFunctionRG

Location : West Europe

Subscription [\(change\)](#) : Azure — dostęp próbny (sponsorowany)

Subscription ID : 444d02f4-b6be-455b-9fc7-5044cc7eb407

Definition : 1 trigger, 2 actions

Status : Enabled

Runs last 24 hours : 0 successful, 0 failed

Integration Account : -- --

Summary

Logic App 

Create

Name *

Subscription

Azure — dostęp próbny (sponsorowany) 

Resource group

AzureLogiAppWithFunctionRG 

Logic App Status

Enabled 

General Best Practices

- Problems with connectors (API connections)
 - Duplicate connections
 - Unused connection
 - Who are using this connection?
- Find orphaned API connectors

General Best Practices

<https://www.integration-playbook.io/docs/find-orphaned-api-connectors>

Find Orphaned API Connectors

Updated On 04 Mar 2020 | 2 Minutes To Read | Contributors   Print  Comments  Share  Dark

How to / Logic Apps / Admin

Filter

Logic Apps

Deployment

Admin

Properly Disable Logic Apps in Powershell

Find Orphaned API Connectors

Check the details of my Logic App SAP connectors

Which Logic App Connector uses which On Premise Data Gateway

Development

Patterns

Testing

Message Transformation

Logic App Integration Accounts

Terraform

Powered by 

One of the pains in Logic App development is that you will add connectors which create a connector resource in Azure but over time as things change you end up with unused API Connections which are no longer used by a Logic App. It is not easy to workout which API Connection is used by which Logic App!

I have put together a powershell script which will look at all of the API Connections in your resource group and then inspect every Logic App in your resource group and check if the API Connections are used. You can then get the csv file that is produced to easily identify orphaned API Connections.

You could also modify the below script to remove those Azure API Connection resources if you wanted using the Remove-AzResource command.

Full Script

Shell

Copy

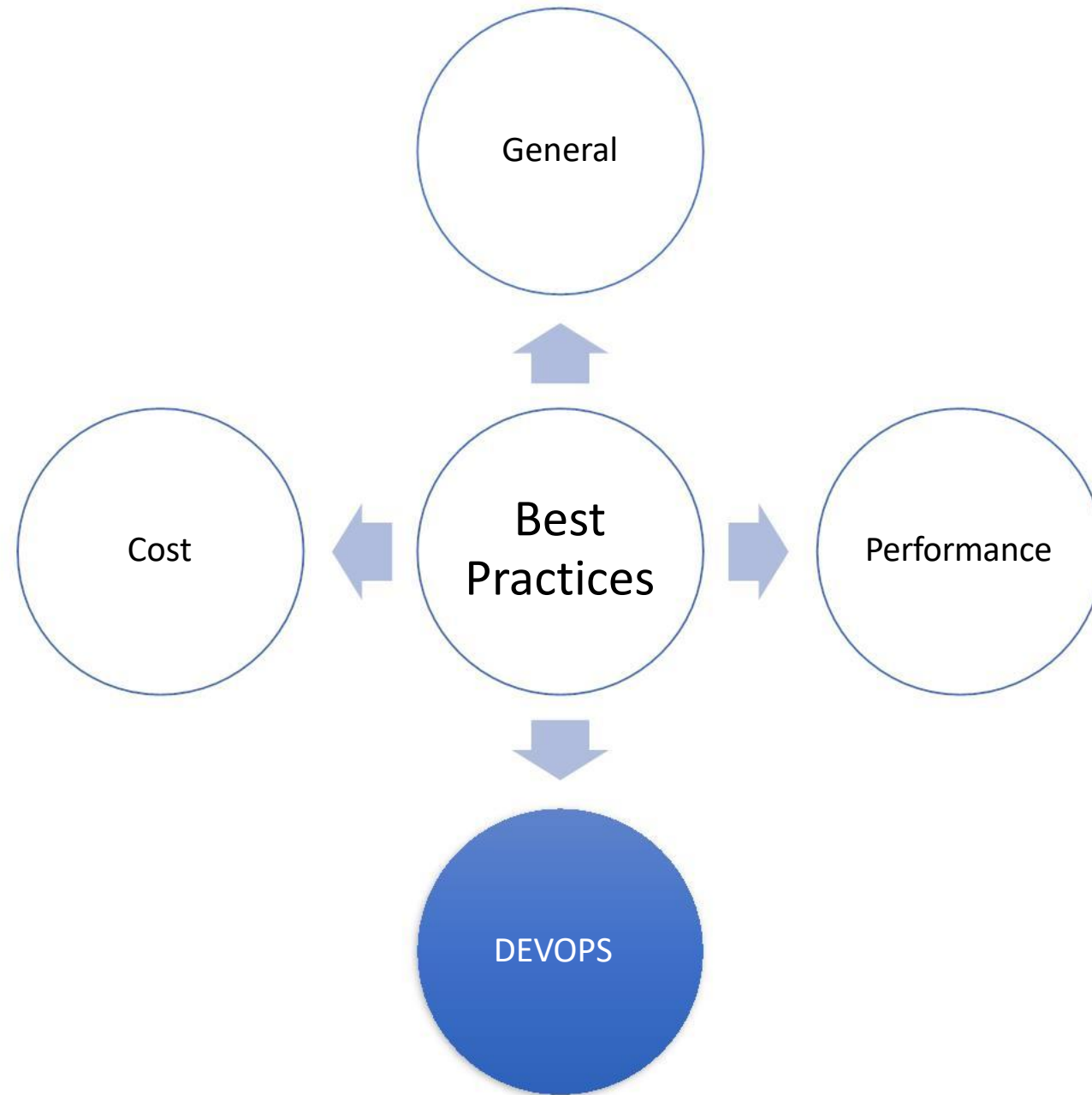
```
#This will check for SAP API Connections and look at where they are pointing to

$subscriptionName = 'My Subscription'
$resourceGroupName = 'My Resource Group'

#use the collection to build up objects for the table
$connectorDictionary = New-Object "System.Collections.Generic.Dictionary`2[System
```

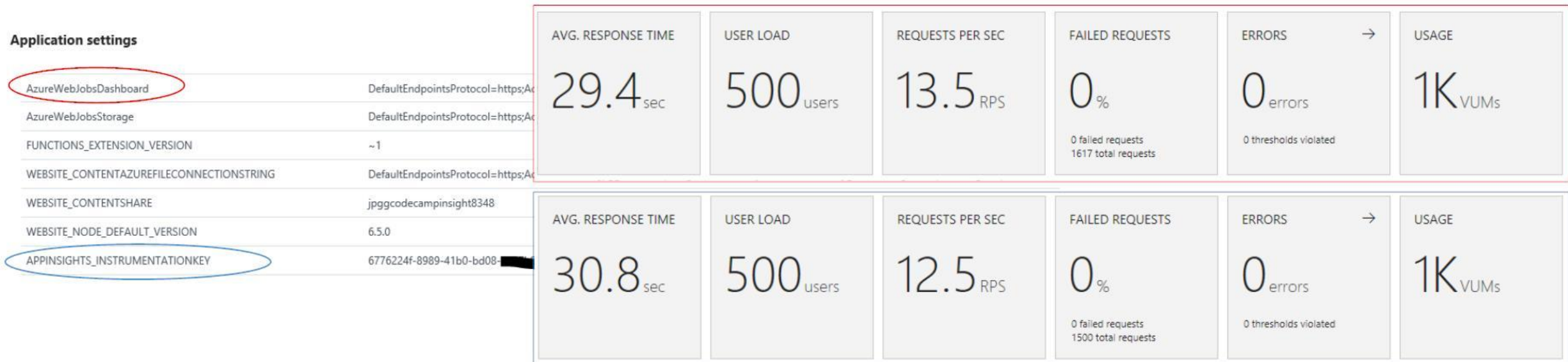
General best practices

- **Avoid long-running functions:**
 - Functions that run for a long time can time out
- **Use queues for cross-function communication:**
 - If you require direct communication, consider Durable Functions or Azure Logic Apps
- **Write stateless functions:**
 - Functions should be stateless
 - State data should be associated with your input and output payloads
- **Code defensively:**
 - Assume that your function might need to continue from a previous failure point



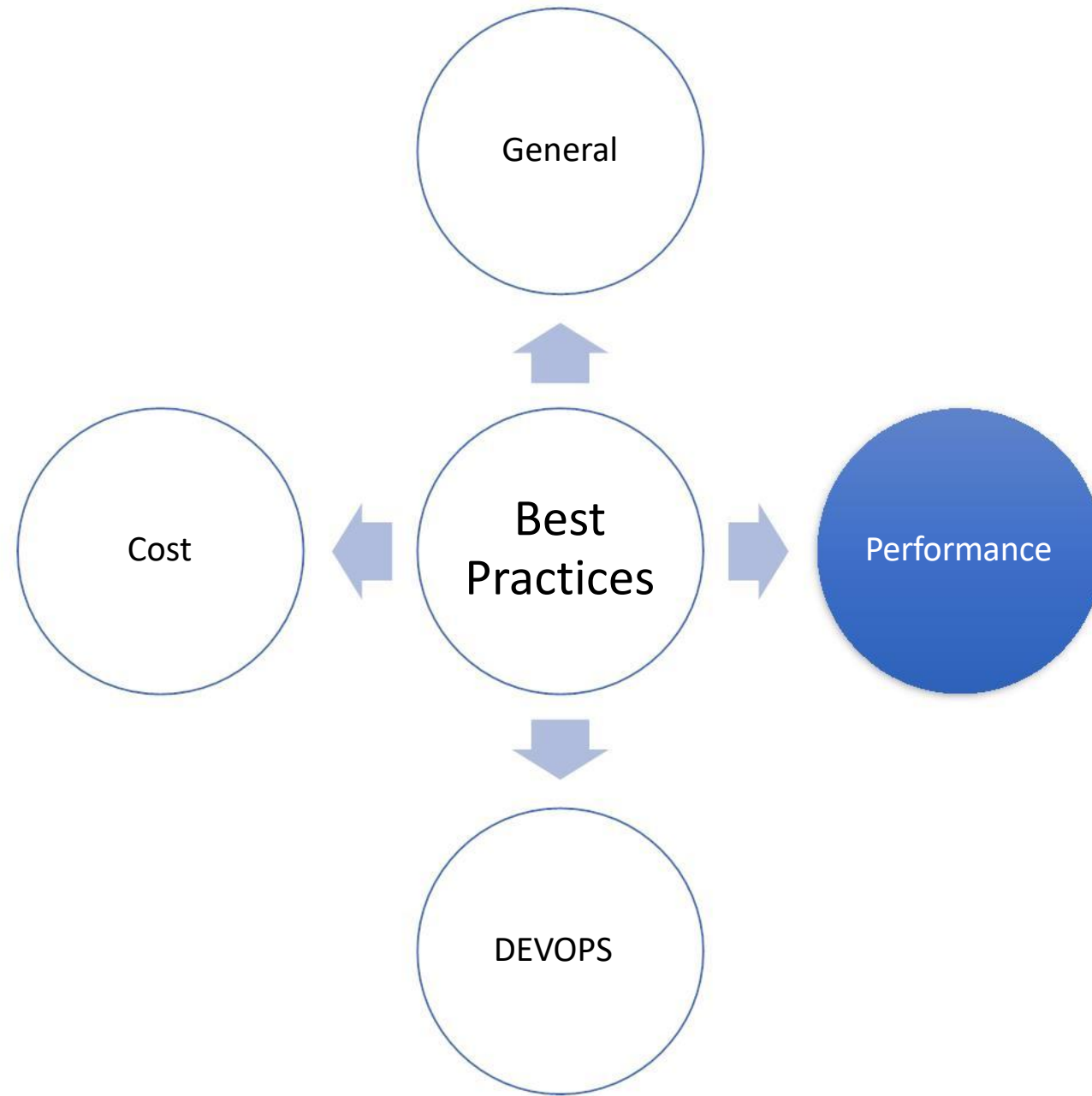
DEV/OPS

- Use **source control** and **CI/CD pipeline**
- Always Monitoring!
 - Azure Functions has two built in monitoring solutions
 - **WebJobs dashboard** & **Application Insights**



DEVOPS / Application Insights

- **Live Stream:** see a near-live view of what's coming from your Function App
- **Metrics Explorer:** insights on your metrics coming from your Function App
- **Failures:** insights on which things are failing
- **Performance:** information on the count, latency, and more of Function executions
- **Servers:** resource utilization and throughput per server (Putting the servers back in Serverless)
- **Analytics Portal:** Write custom queries against your data
- **Alerts***



Scale and hosting Function Apps

- You can choose between three types of plans:
 - **Consumption:**
 - Instances are dynamically instanced, and you are charged based on compute time
 - **Premium**
 - Instances of the Azure Functions host are added and removed based on the number of incoming events just like the Consumption plan, but provides additional features like: VNet connectivity; unlimited execution duration; and more predictable pricing
 - **App Service plan:**
 - Traditional App Services model used with Web Apps, API Apps, and Mobile Apps
- The type of plan controls:
 - How host instances are scaled out
 - The resources that are available to each host

Operating system

The Operating System has been recommended for you based on your selection of runtime stack.

Operating System *

Plan

The plan you choose dictates how your app

Plan type * ⓘ

Consumption (Serverless)

Premium

App service plan

Consumption (Serverless)

Scalability Best Practices

- Receive messages in batch whenever possible/the functions process messages in batches

```
{
  "queues": {
    // The maximum interval in milliseconds between
    // queue polls. The default is 1 minute.
    "maxPollingInterval": 2000,
    // The visibility timeout that will be applied to messages that fail processing
    // (i.e. the time interval between retries). The default is zero.
    "visibilityTimeout": "00:00:30",
    // The number of queue messages to retrieve and process in
    // parallel (per job function). The default is 16 and the maximum is 32.
    "batchSize": 1,
    // The number of times to try processing a message before
    // moving it to the poison queue. The default is 5.
    "maxDequeueCount": 5,
    // The threshold at which a new batch of messages will be fetched.
    // The default is batchSize/2. Increasing this value will increase the
    // level of concurrency and therefore throughput. New batches of messages
    // will be pulled until the number of messages being processed is greater
    // than this threshold. When the number dips below this threshold, new
    // batches will be fetched.
    "newBatchThreshold": 1
  }
}
```

Scalability Best Practices

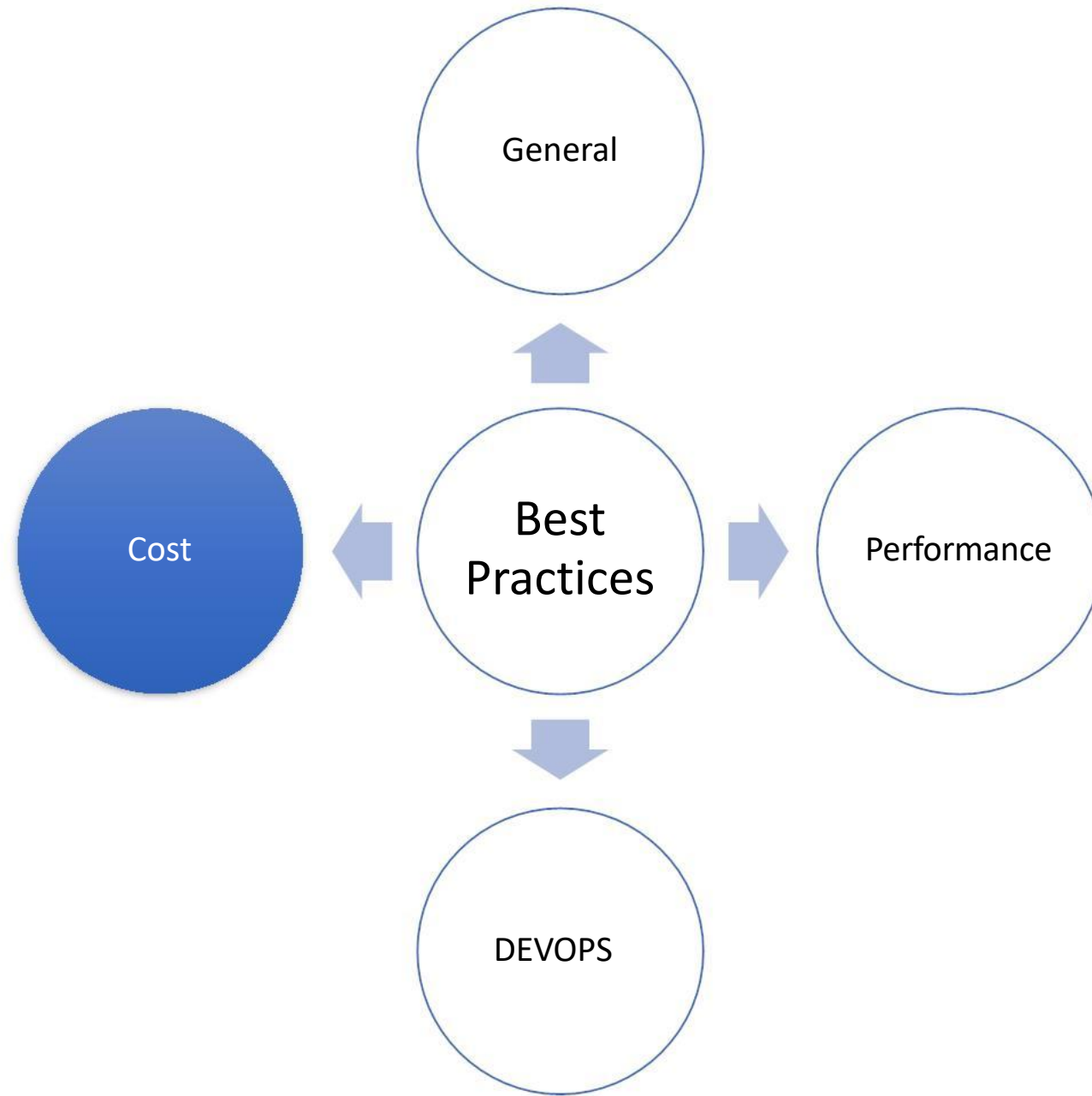
- Use async code but avoid blocking calls
 - C# example: `Result` V/S `await`
- Watch your outbound socket connections
 - limit is 300 on Consumption plan + use **connection pooling**

Scalability Best Practices

- Configure host behaviors to better handle concurrency

```
{  
  "http": {  
    "routePrefix": "api",  
    "maxOutstandingRequests": 20,  
    "maxConcurrentRequests": 10,  
    "dynamicThrottlesEnabled": false  
  }  
}
```

“scale paradox?”



Cost Function Apps

- Consumption plan
 - **Executions**
 - total number of requested executions each month for all functions
 - The first million executions are included **free each month**.
 - **Resource consumption**
 - “**Observed resource consumption**” measured in gigabyte seconds (GB-s).
 - Observed resource consumption is calculated by multiplying average memory size in gigabytes by the time in milliseconds it takes to execute the function.
 - Memory used by a function is measured by rounding up to the nearest 128 MB, up to the maximum memory size of 1,536 MB
 - Execution time calculated by rounding up to the nearest 1 ms.
 - The minimum execution time and memory for a single function execution is 100 ms and 128 mb respectively.
 - Functions pricing includes a **monthly free** grant of 400,000 GB-s.
- App Service plan
 - Same as usual for Web apps

Cost Logic Apps

Plan

The plan you choose dictates how your app scales, what features are enabled, and how it is priced. [Learn more](#)

Plan type * ⓘ

Windows Plan (Central US) ⓘ

Premium ^

Premium

App service plan

 Microsoft Azure

[Przegląd](#) [Rozwiązania](#) [Produkty](#) [Dokumentacja](#) [Cennik](#) [Szkolenia](#) [Portal Marketplace](#) [Partnerzy](#) [Pomoc techniczna](#) [Blog](#) [Więcej](#)

Region:
Niemcy Północne (publiczna)

Waluta:
Euro (€)

[Kontakt ze sprzedażą](#) [Wyszukiwanie](#)

Szczegóły cennika

Za każdym razem, gdy definicja aplikacji logiki uruchamia wyzwalacze, wykonania akcji i łączników są mierzone.

CENA ZA WYKONANIE	
Akcje	€0,000034
Łącznik standardowy	€0,000172
Łącznik przedsiębiorstwa	€0,001375

Loader

<https://loader.io/>

Loader.io is a FREE load testing service that allows you to stress test your web-apps & api's with thousands of concurrent connections.

Function Apps

▼ ⚡ loaderio



► Functions (Read Only)

▼ Proxies (Read Only)

↪ loaderio-verifier

► Slots (preview)

loaderio-verifier

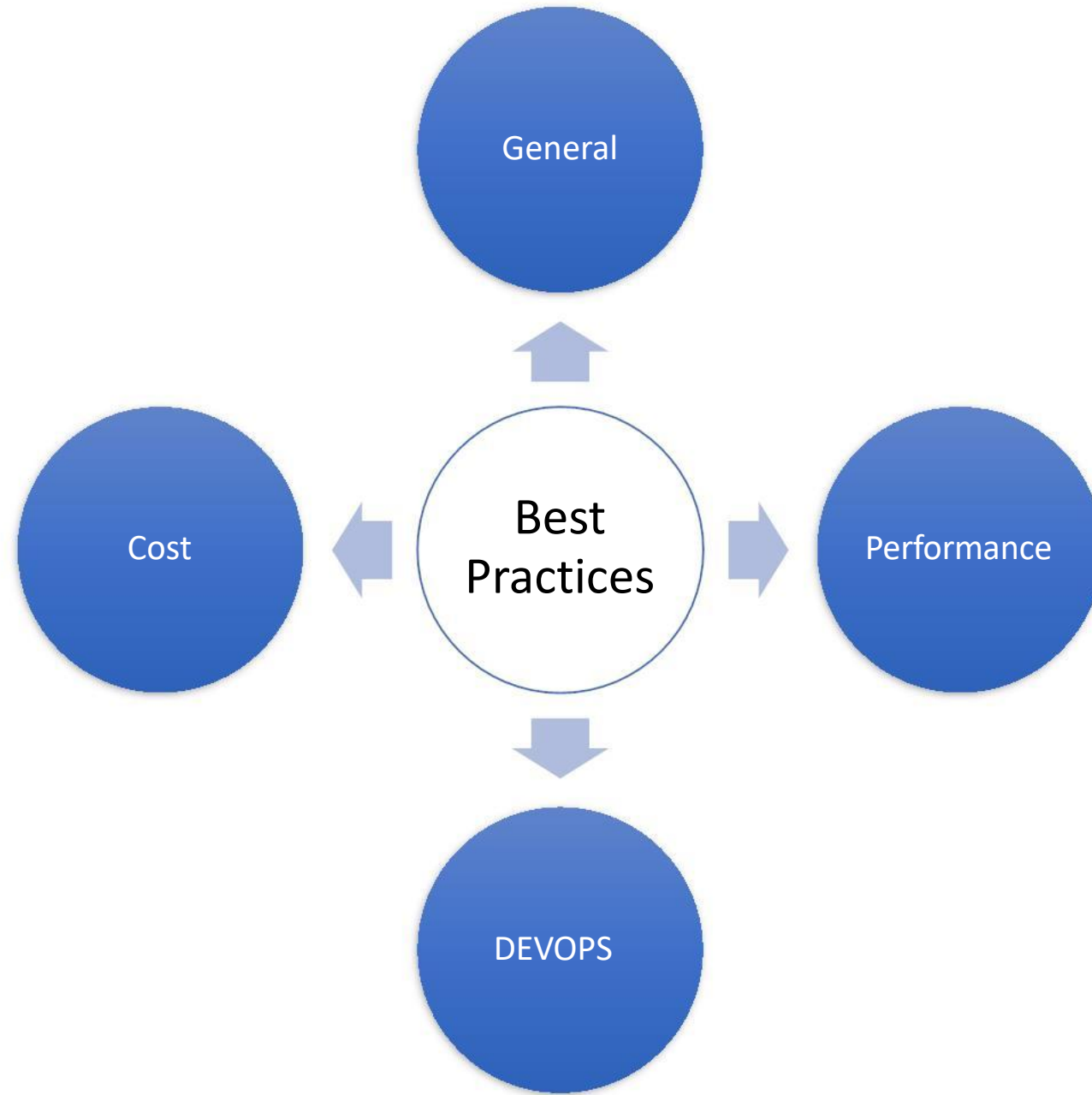
Your app is currently in read only mode because you are running from a package file. To make

Proxy URL

`https://loaderio.azurewebsites.net/loaderio-6d6c3d22ae7ecbbda54e74c8780522ee`

Route template

`/loaderio-6d6c3d22ae7ecbbda54e74c8780522ee`



DEMO 3

References

- Optimize the performance and reliability of Azure Functions
 - <https://docs.microsoft.com/en-us/azure/azure-functions/functions-best-practices>
- <https://stackoverflow.com/questions/61101593/azure-logic-app-and-function-app-performance-difference>
 - <https://stackoverflow.com/questions/61101593/azure-logic-app-and-function-app-performance-difference>
- Azure Functions – Significant Improvements in HTTP Trigger Scaling
 - <https://www.azurefromthetrenches.com/azure-functions-significant-improvements-in-http-trigger-scaling/>
- Azure Function Apps: Performance Considerations
 - <https://blogs.msdn.microsoft.com/amitagarwal/2018/04/03/azure-function-apps-performance-considerations/>
- Host Health Monitor
 - <https://github.com/Azure/azure-functions-host/wiki/Host-Health-Monitor>
- Functions pricing
 - <https://azure.microsoft.com/en-us/pricing/details/functions/>
- Logic apps pricing
 - <https://azure.microsoft.com/en-us/pricing/details/logic-apps/>

References

- Best practices for Azure Functions

<https://docs.microsoft.com/en-us/azure/azure-functions/functions-best-practices>

- Processing 100,000 events per second on Azure Functions

<https://azure.microsoft.com/pl-pl/blog/processing-100-000-events-per-second-on-azure-functions/?ref=msdn>

- Serverless samples

<https://www.serverlesslibrary.net/sample>

- Azure Functions – Prepare for continuous delivery

<https://blogs.msdn.microsoft.com/visualstudioalmrangers/2017/09/06/azure-functions-prepare-for-continuous-delivery/>

- High throughput Azure Functions on HTTP

<https://codez.deedx.cz/projects/high-throughput-azure-functions-on-http/>

Q & A

Email: info@zalnet.pl

Thank you for your precious time 😊